ASSIGNMENT 5 – CO5

PART A | TASK A2

DON’T READ WHAT YOU DON’T NEED

Buffered I/O Design for a 50 MB Attendance Log

Course: Programming in Java
Topic: Data Persistence Engineering – Files and Databases

Design report covering the predicted cost of byte-at-a-time reading, a buffered redesign, limitations of buffering, and comparison with Files.readAllLines().

1. Problem Understanding

The question describes a program that reads a 50 MB attendance log one byte at a time using FileInputStream.read() to count the number of lines. The design task is to predict the number of read calls, explain the performance problem, redesign the operation using buffering, and identify a case where buffering provides little benefit.

2. A2.1 – Predict the Cost

Assuming 50 MB means 50 × 1,024 × 1,024 bytes, the file contains 52,428,800 bytes. A loop that invokes read() once for each byte therefore makes approximately 52,428,800 Java read() calls to consume the entire file (plus a final call that detects end-of-file, depending on the loop structure). If MB is interpreted as decimal, the corresponding count is 50,000,000 bytes.

Why it is slow

The program performs a method call for every byte. With an unbuffered FileInputStream, single-byte reads can result in very frequent native I/O operations. Each operation has call overhead and repeated transitions between Java and the operating system. The file may be served partly from the operating-system page cache, so it is not correct to claim that every call necessarily causes a physical disk access; nevertheless, the very high call frequency creates avoidable overhead. The algorithm still has O(N) time complexity, where N is the number of bytes, but its constant overhead is large.

3. A2.2 – Redesign Using Buffering

Wrap FileInputStream in BufferedInputStream, or read blocks into a byte array. A buffer temporarily stores a block of bytes in memory. The program then processes those bytes from memory instead of requesting one byte from the underlying stream for every iteration.

Approach	Approximate underlying refill/read operations	Extra memory
One byte at a time, unbuffered	Up to about N single-byte reads (N = file bytes)	O(1)
Buffered, buffer size B	About ceil(N / B) buffer refills	O(B)

For example, with a 8 KB (8,192-byte) buffer, a 50 MiB file needs approximately 6,400 buffer refills rather than more than 52 million one-byte read requests. The exact number depends on the buffer size and stream implementation. This reduces the frequency of underlying I/O calls substantially. The program still examines each byte once, so the overall time complexity remains O(N), while auxiliary memory is O(B).

SIMATS-style pseudocode: buffered line counter

```
ALGORITHM COUNT_LINES_BUFFERED(filePath)
    lineCount <- 0
    previousByteWasCarriageReturn <- FALSE

    OPEN filePath using FileInputStream
    WRAP FileInputStream with BufferedInputStream

    WHILE nextByte can be read
        IF nextByte equals LINE_FEED THEN
            lineCount <- lineCount + 1
            previousByteWasCarriageReturn <- FALSE
        ELSE IF nextByte equals CARRIAGE_RETURN THEN
            lineCount <- lineCount + 1
            previousByteWasCarriageReturn <- TRUE
        ELSE
            previousByteWasCarriageReturn <- FALSE
        ENDIF
```

```
    ENDWHILE

    CLOSE BufferedInputStream
    RETURN lineCount
END ALGORITHM
```

Line-ending note

The pseudocode above treats LF (\n) and CR (\r) as line terminators. If the file uses Windows CRLF pairs, this simple version counts the pair as two terminators. A production implementation should treat CRLF as one line ending by remembering whether the previous byte was CR, and count a CR only when it is not followed by LF. The same care is needed for a final line that has no terminating newline: whether it counts as a line depends on the application's definition of a line.

A Java implementation can use BufferedInputStream and count newline bytes. However, Task A2 is in Part A – Persistence Design & Pseudocode. The assignment asks for a redesign and explanation, not a Java program. Java implementation and measured execution belong to Part B tasks.

4. A2.3 – When Buffering Does Not Help

Buffering gives little or no benefit when the program already reads data in large blocks, or when the workload is dominated by processing rather than I/O. For example, if a program already uses a large byte-array read to fetch tens of kilobytes at a time, wrapping it in another buffer may add an extra memory copy without meaningfully reducing the number of underlying read calls. Buffering can also be less useful for a tiny file that is read only once, because setup overhead may outweigh the small amount of I/O saved.

5. Alternative Concept – Read the Whole File with Files.readAllLines()

The alternative is to load all lines into a List in one operation using Files.readAllLines(path). The program can then count the number of entries in the returned list. This is convenient and simple when the file is small enough to fit comfortably in memory.

Complexity and memory comparison

Method	Time complexity	Memory	Suitable use
Buffered stream	O(N)	O(B), plus small counters/state	Large files; process incrementally; limited memory
Files.readAllLines()	O(N)	O(N) overall, including line objects and strings	Small/moderate files that comfortably fit in memory

For a 50 MB file, Files.readAllLines() requires memory for the complete contents plus the List, individual String objects and their internal character storage. Actual heap usage can be significantly greater than the file's byte size. It may cause high memory pressure or an OutOfMemoryError when the file is large or the available heap is limited. A buffered stream processes the file incrementally and does not retain every line, so it is generally the safer choice for a large attendance log.

6. Overall Comparison and Decision

For the 50 MB attendance log, use BufferedInputStream (or a suitable buffered reader if the file is decoded as text) and process the input incrementally. Both buffered processing and Files.readAllLines() have O(N) time complexity, but their memory requirements differ: buffered processing uses O(B) auxiliary memory, whereas readAllLines() uses O(N) memory. The one-byte unbuffered approach also has O(N) time complexity but incurs much greater per-read overhead.

7. Conclusion

A byte-at-a-time scan of a 50 MiB file can make approximately 52.4 million read calls. Using a buffer reduces underlying read/refill operations to roughly the file size divided by the buffer size while preserving O(N) processing time. Buffering is the appropriate design for a large log because it combines lower I/O-call overhead with bounded memory. Files.readAllLines() is convenient for smaller files but retains the complete file in memory.

Submission note

Task A2 is in Part A – Persistence Design & Pseudocode. No Java source code or execution screenshot is required for this task. The report provides the requested analysis and algorithmic pseudocode; implementation and timing measurements are addressed in Part B.